

# AfriMarkets

Technical Documentation · R Package v1.0.5

Access · Analyze · Understand African Capital Markets

Olabiyl Aurel Géoffroy Odjo & Koffi Fredy Sessie

June 2026 · MIT License · GitHub: [Koffi-Fredysessie/AfriMarkets](https://github.com/Koffi-Fredysessie/AfriMarkets)

---

# 1. Project Overview

AfriMarkets is an open-source R package that provides unified, programmatic access to historical and real-time financial data from African stock exchanges. Designed for researchers, data scientists, analysts, and developers, it bridges the gap between the richness of African capital markets and the global data-science community by offering a single consistent API across heterogeneous exchanges and data sources.

## Key Capabilities

| Feature                 | Description                                                         |
|-------------------------|---------------------------------------------------------------------|
| Ticker discovery        | Browse all available indexes and shares per market                  |
| Historical OHLCV data   | Fetch daily Open/High/Low/Close/Volume across any custom date range |
| Flexible output formats | Long format, wide format, or both at once                           |
| Interactive charts      | Candlestick and multi-ticker line charts via highcharter            |
| Performance KPIs        | Return, volatility, Sharpe ratio, max drawdown, and more            |
| Statistical tests       | Normality and stationarity test batteries                           |
| Simple API              | Consistent function signatures across all supported markets         |

Supported R version: R ≥ 4.1.0 · License: MIT

## Authors & Roles

| Name                        | Role                 | Contact                        |
|-----------------------------|----------------------|--------------------------------|
| Olabiyl Aurel Géoffroy Odjo | Creator & Maintainer | odjoaurel@gmail.com            |
| Koffi Fredy Sessie          | Author               | koffisessie@gmail.com          |
| Abdoul Oudouss Diakité      | Author               | abdouloudoussdiakite@gmail.com |
| Steven P. Sanderson II, MPH | Author               | spsanderson@gmail.com          |

---

## 2. Installation

AfriMarkets is currently available as a development version hosted on GitHub. An active internet connection is required both at install time (to resolve dependencies) and at runtime (to fetch live market data).

### Installation from GitHub

```
# Install devtools if needed
install.packages("devtools")

# Install AfriMarkets
devtools::install_github("Koffi-Fredysessie/AfriMarkets")
```

Note: A CRAN release is planned once the package reaches stable lifecycle status. The current version (1.0.5) is tagged as experimental — the API may evolve.

### Dependencies

| Category            | Packages                                       |
|---------------------|------------------------------------------------|
| Data wrangling      | dplyr, tidyr, tibble, purrr, magrittr, stringr |
| HTTP & web scraping | httr, httr2, rvest, jsonlite                   |
| Time series & dates | xts, lubridate, tseries                        |
| Statistics          | nortest, fBasics, goftest, stats               |
| Visualisation       | highcharter                                    |
| OOP & utilities     | methods, rlang                                 |

### 3. Architecture & Market Registry

#### 3.1 The S4 Class: african\_market

AfriMarkets uses the S4 object-oriented system (package methods) to represent each stock exchange as a structured, typed object. The central class is `african_market`, instantiated by `GET_tickers()` for every supported exchange.

| Slot         | Type       | Description                                |
|--------------|------------|--------------------------------------------|
| @Market      | character  | Exchange code (e.g. 'BRVM', 'NGX')         |
| @Shares      | data.frame | Table of all listed equities with metadata |
| @Indexes     | data.frame | Table of all market indexes                |
| @ListShares  | character  | Character vector of equity ticker codes    |
| @ListIndexes | character  | Character vector of index codes            |

#### Accessing slots

```
brvm <- GET_tickers("BRVM") # returns an african_market S4 object
brvm@Shares # data.frame of all shares
brvm@Indexes # data.frame of all indexes
brvm@ListShares # character vector of ticker codes
```

#### 3.2 Market Registry (.MARKET\_REGISTRY)

The package maintains an internal registry (`.MARKET_REGISTRY` defined in `GET_tickers.R`) that maps each exchange code to its scraping backend, base URL, and parsing logic. Adding a new exchange requires only one entry in this registry — no other source files need modification.

| Code | Exchange                                | Region              | Backend           |
|------|-----------------------------------------|---------------------|-------------------|
| BRVM | Bourse Régionale des Valeurs Mobilières | UEMOA (8 countries) | Custom scraper    |
| GSE  | Ghana Stock Exchange                    | Ghana               | Custom scraper    |
| NGX  | Nigerian Exchange Group                 | Nigeria             | Investing.com API |
| JSE  | Johannesburg Stock Exchange             | South Africa        | Investing.com API |
| MSE  | Casablanca Stock Exchange               | Morocco             | Investing.com API |
| EGX  | Egyptian Exchange                       | Egypt               | Investing.com API |
| TSE  | Tunis Stock Exchange                    | Tunisia             | Investing.com API |

Coming soon: Kenya NSE · Botswana BSE · Mauritius SEM

#### 3.3 Data Registry (.DATA\_REGISTRY)

A parallel internal registry (`.DATA_REGISTRY` in `GET_data.R`) maps each exchange code to its data-fetching routine, date-range logic, and output normalisation. Both registries must be updated

together when adding a new market.

## 4. Core Function Reference

### 4.1 GET\_tickers(market\_code)

Queries the market registry and returns an S4 african\_market object for a single exchange, or a named list of african\_market objects when market\_code = 'ALL'.

GET\_tickers(market\_code)

| Argument    | Type      | Values                                          | Description                                                             |
|-------------|-----------|-------------------------------------------------|-------------------------------------------------------------------------|
| market_code | character | 'BRVM','GSE','NGX','JSE','MSE','EGX','SE','ALL' | Code of the target exchange, or 'ALL' to retrieve all supported markets |

Returns: S4 african\_market (single market) or named list (ALL)

```
# Single market
brvm <- GET_tickers("BRVM")

# All markets
all <- GET_tickers("ALL")
all$NGX@Shares
```

### 4.2 GET\_data()

Downloads historical OHLCV data for one or multiple tickers from a given exchange, over a specified date range, with configurable output format.

GET\_data(market\_code, ticker, from, to, output\_format)

| Argument      | Type      | Default       | Description                                  |
|---------------|-----------|---------------|----------------------------------------------|
| market_code   | character | —             | Exchange code (BRVM, NGX, JSE, etc.)         |
| ticker        | character | —             | Ticker symbol, vector of symbols, or keyword |
| from          | character | Sys.Date()-90 | Start date 'YYYY-MM-DD'                      |
| to            | character | Sys.Date()    | End date 'YYYY-MM-DD'                        |
| output_format | character | 'by_col'      | 'by_col', 'by_row', or 'all'                 |

#### Ticker keywords

| Keyword       | Returns                            |
|---------------|------------------------------------|
| "ALL"         | All instruments (shares + indexes) |
| "ALL SHARES"  | Equities only                      |
| "ALL INDEXES" | Indexes only                       |

#### Output format reference

| output_format      | Structure                               | Best for                                     |
|--------------------|-----------------------------------------|----------------------------------------------|
| "by_col" (default) | Long/tidy — one row per date per ticker | dplyr pipelines, ggplot2, time-series models |

| output_format | Structure                    | Best for                                     |
|---------------|------------------------------|----------------------------------------------|
| "by_row"      | Wide — tickers as columns    | Correlation matrices, portfolio optimisation |
| "all"         | Named list with both formats | Exploratory analysis                         |

```

# Single ticker – last 90 days
df <- GET_data("BRVM", ticker = "BICC")

# Multiple tickers – custom range
df <- GET_data("BRVM", ticker = c("BICC", "BOAB"), from = "2023-01-01")

# Wide format
df_wide <- GET_data("NGX", ticker = "ALL SHARES", output_format = "by_row")

```

### 4.3 pplot()

Renders an interactive financial chart using highcharter. Accepts either a raw data.frame (output of GET\_data()) or a market code combined with a ticker vector. Single-ticker calls produce a candlestick chart with colour-coded volume bars; multi-ticker calls produce a grouped line chart.

```
pplot(market, stock, from, to, up.col, down.col)
```

| Argument | Default    | Description                                             |
|----------|------------|---------------------------------------------------------|
| market   | —          | Exchange code (character) or data.frame from GET_data() |
| stock    | —          | Ticker or character vector of tickers                   |
| from     | —          | Start date 'YYYY-MM-DD'                                 |
| to       | Sys.Date() | End date 'YYYY-MM-DD'                                   |
| up.col   | "green"    | Colour for bullish candles / positive line              |
| down.col | "red"      | Colour for bearish candles / negative line              |

```
# Candlestick chart
pplot(market = "BRVM", stock = "BICC", from = "2023-01-01")

# Multi-ticker line chart
pplot(market = "BRVM", stock = c("BICC", "BOAB", "SNTS"), from = "2023-01-01")

# From data.frame
pplot(market = df)
```

### 4.4 normality\_test(x)

Tests whether a numeric price (or return) series follows a normal distribution. Useful before applying parametric financial models. Returns a named list of p-values.

```
normality_test(x) # x: numeric vector or ts object
```

### 4.5 stationarity\_test(x, type.test)

Runs one or more unit-root and stationarity tests on a numeric series. type.test matching is case-insensitive; unknown names are dropped with a diagnostic message. Pass 'ALL' to run the full test battery.

| Test                          | Null Hypothesis                    | Min. Obs. |
|-------------------------------|------------------------------------|-----------|
| Box-Pierce / Ljung-Box        | No autocorrelation                 | 3         |
| KPSS                          | Series is stationary               | 3         |
| Augmented Dickey-Fuller (ADF) | Unit root present (non-stationary) | 7         |
| Phillips-Perron               | Unit root present (non-stationary) | 4         |

```
stationarity_test(df$Close, type.test = "ALL")
stationarity_test(df$Close, type.test = "augmented dickey-fuller test (adf)")
```

## 4.6 RECOVER\_last\_download()

Returns the result of the most recent GET\_data() call from the session cache. Useful for recovering data after an accidental variable overwrite without making an extra network request.

```
last <- RECOVER_last_download()
```

---

## 5. Financial Performance Indicators

AfriMarkets computes the following KPIs over any selected date range. The Sharpe ratio assumes a risk-free rate of zero.

| Indicator                 | Formula                                                 | Interpretation                        |
|---------------------------|---------------------------------------------------------|---------------------------------------|
| Total Return (%)          | $(P_{end} / P_{start} - 1) \times 100$                  | Absolute gain/loss over the period    |
| Annualised Return (%)     | $(P_{end}/P_{start})^{(252/n)} - 1$                     | Return normalised to 252 trading days |
| Annualised Volatility (%) | $sd(\log \text{ returns}) \times \sqrt{252} \times 100$ | Risk — dispersion of daily returns    |
| Sharpe Ratio              | $\text{mean}(\log r) / sd(\log r) \times \sqrt{252}$    | Risk-adjusted return ( $r_f = 0$ )    |
| Max Drawdown (%)          | $\min((P - \text{cummax}(P)) / \text{cummax}(P))$       | Worst peak-to-trough decline          |
| Max / Min Price           | $\max(\text{High}) / \min(\text{Low})$                  | Price range over the period           |
| Average Volume            | $\text{mean}(\text{Volume})$                            | Liquidity indicator                   |

---

## 6. Contributing to AfriMarkets

---

### 6.1 General Workflow

All contributions — bug fixes, new features, documentation improvements — follow the standard GitHub fork-and-pull-request workflow. Please open an issue first to discuss major changes before submitting a PR. Commit messages must follow Conventional Commits style.

```
# 1. Fork the repo and clone it
git clone https://github.com/Koffi-Fredysessie/AfriMarkets.git

# 2. Create a feature branch
git checkout -b feature/add-xyz-market

# 3. Commit (Conventional Commits style)
git commit -m "feat: add XYZ market support"

# 4. Push and open a Pull Request
git push origin feature/add-xyz-market
```

### 6.2 Adding a New Market (2 steps)

The registry-driven architecture means adding a new exchange requires changes in exactly two places — no other files need to be touched.

| Step | File            | What to add                                                                              |
|------|-----------------|------------------------------------------------------------------------------------------|
| 1    | R/GET_tickers.R | .MARKET_REGISTRY entry: exchange code, name, region, scraper type, base URL              |
| 2    | R/GET_data.R    | .DATA_REGISTRY entry: matching exchange code, data-fetch function, date logic, output no |

Always run `devtools::check()` and `testthat` tests locally before pushing.

### 6.3 Adding a New Feature or Function

#### Create the .R file

Place your function in the R/ directory. One function per file is recommended for maintainability.

#### Write Roxygen2 documentation

Add `@title`, `@description`, `@param`, `@return`, `@export` and `@examples` tags above the function. Run `devtools::document()` to regenerate `man/` files.

#### Export the function

Ensure `@export` is present in the Roxygen block so the function is listed in `NAMESPACE`.

#### Write unit tests

Add a test file under `tests/testthat/`. Use `testthat::expect_*` assertions covering normal use, edge cases, and error conditions.

#### Update vignettes if needed

If the feature changes user-facing behaviour, update the relevant vignette in `vignettes/`.

#### Run full check

devtools::check() must pass with 0 errors and 0 warnings before submitting a PR.

## 6.4 Bug Reports

Open an issue at [github.com/Koffi-Fredysessie/AfriMarkets/issues](https://github.com/Koffi-Fredysessie/AfriMarkets/issues). Include: R version (R.version), package version (packageVersion('AfriMarkets')), full error message or unexpected output, and a minimal reproducible example.

---

# 7. Use Cases

| Domain               | Application                                                                       |
|----------------------|-----------------------------------------------------------------------------------|
| Academic research    | Market efficiency, volatility clustering, cointegration across African exchanges. |
| Portfolio management | Cross-market diversification, risk budgeting, correlation analysis.               |
| Econometrics         | Unit root tests, GARCH modelling, event studies on African equities.              |
| Quantitative finance | Factor models, backtesting systematic strategies.                                 |
| Data journalism      | Visualising African market trends with interactive highcharter charts.            |
| Teaching             | Real market data for financial analysis courses in African universities.          |